

Submission #5: Simple Search

Prepared by

Ivan Hromovyi

in

IKT203

Algorithms and Data Structures

Department of Information and Communication Technology

Faculty of Engineering and Science

September 18, 2026

Contents

1 Report **1**

1.1 `TLinkedList` implementation 1

1.2 *O(n)* complexity in `Find()` method 1

1.3 `Every()` vs `PrintEveryAccountInDateRange()` 1

List of Figures

1 `Find()` results 1

1 Report

Initially the assignment was focused on developing `Simple Bank Account Application` that follows various searching strategies. Main goal was to implement and understand algorithm designs, pointer manipulations, memory ownership and performance analysis.

1.1 `TLinkedList` implementation

The choice fell on using a doubly linked list over a singly linked or circular list. The doubly linked design was chosen for the generality of the TLinkedList class itself. The *prev* pointer makes it possible for `GetAtIndex()` function to traverse from whichever end is closer to the target index, and `Remove()` function is able to unlink a node in `O(1)` without a separate scan to locate its predecessor.

1.2 *O(n)* complexity in `Find()` method

The `Find()` function has *O(n)* time complexity and and *O(1)* auxiliary space. for an element near the end of the list and for a non-existent element, to demonstrate the time complexity that follows from this. The search stops when the function finds the first match object and the comparisons stop at the exact moment. To make this even clearer, an additional case was added: searching for an element near the front of the list. The results are shown in Figure 1.

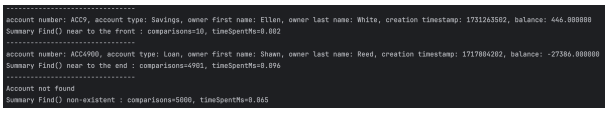


Figure 1: `Find()` results

The screenshot proves that the comparisons stop and the time between the the first case and the second change dramatically. This is a clean linear progression and is the clearest evidence of *O(n)* time complexity in this implementation. The difference in time between the third and the second cases is probably due to the measurement noise.

1.3 `Every()` vs `PrintEveryAccountInDateRange()`

The key difference between `Every()` method and `PrintEveryAccountInDateRange()` function is that the `Every()` method is open for modifications, it extracts the condition out and takes it as a parameter. For instance, in this assignment `Every()` function was used to generate a list based on the month of the year, but it is also possible to use this function with other keys such as an account number, balance, etc. However, the cost of it is a type safety, because the compiler cannot verify a callback is being paired with the right kind of key.

On the other hand, there is `PrintEveryAccountInDateRange()` function, where the signature is fixed and it is impossible to call it with the wrong shape of argument. This way, it trades flexibility for that guarantee.